The Token Tax on Indian Languages: Measuring Tokenizer Premiums Across All 22 Scheduled Languages and 14 Tokenizers

Divij Karanam
Independent Researcher

***Abstract*—Large language models (LLMs) process text as subword tokens, and the number of tokens a text requires determines API cost, usable context length and generation latency. Tokenizers built from English-dominated data segment other languages into more tokens, imposing a "token tax". We measure this tax for all 22 scheduled languages of India using 1,024 professionally translated parallel sentences (IN22-Gen) and 14 tokenizers, including GPT-4o, Llama 4, Gemma 3, DeepSeek-V3, Qwen 3 and the Indian-built Sarvam-1, Krutrim-2, IndicBERT v2 and MuRIL. Relative to English, GPT-4o needs 3.10× as many tokens on average (1.55× for Hindi, 13.2× for Santali), and the most efficient LLM tokenizer, Gemma 3, needs 2.33×. Newer tokenizers roughly halve the premium of their predecessors, but not for the scripts with the least web presence. Sarvam-1 is the most efficient LLM tokenizer on its ten target languages (1.28×) yet less efficient than GPT-4o on the remaining twelve. Using human romanizations from the Dakshina dataset, we find that romanized input is cheaper only for older, English-centric tokenizers and costs 38–44% more on Indic-focused ones. Finally, Unicode NFC normalization changes token counts by between −17.4% and +7.6% depending on the tokenizer. We also document a library-level discrepancy that inflated GPT-4o counts by about 46% and describe the verification protocol that caught it. Code and data are released.**

***Index Terms*—tokenization, multilingual NLP, Indian languages, large language models, fairness, romanization**

I. Introduction

Subword tokenizers based on byte-pair encoding or unigram language models [1], [2] map text to the units an LLM reads and generates. Because most commercial APIs bill per token, and context windows and decoding time are also counted in tokens, a language that needs more tokens to express the same content is more expensive, fits less text into context and is slower to generate. Prior work has shown that this "tokenization premium" varies widely across the world's languages [3], [4].

India has 22 languages listed in the Eighth Schedule of its Constitution, written in at least 13 scripts. Several are spoken by tens of millions of people, while others, such as Santali (Ol Chiki script) and Manipuri (Meitei Mayek script), have a very small web footprint. Recent tokenizers have greatly expanded their vocabularies, and several Indian organizations have released models designed for Indian languages. This raises a practical question: *how large is the token tax on Indian languages today, and who pays the most?*

This paper makes four contributions:

1) A measurement of the token premium for all 22 scheduled languages on a professionally translated parallel corpus, across 14 current tokenizers, including four Indian-built ones (Section IV).
2) A verification protocol for token counting, which revealed that loading a community conversion of GPT-4o's tokenizer through a popular library over-counted tokens by about 46% (Section III-D).
3) An analysis of romanized versus native-script input on human-written romanizations (Section IV-E).
4) A measurement of how Unicode normalization changes token counts (Section IV-F).

II. Related Work

Petrov et al. [3] defined the tokenization premium on parallel FLORES-200 sentences and showed premiums above 10× for some languages. Ahia et al. [4] analyzed the resulting cost and utility disparities for commercial APIs. Rust et al. [5] linked tokenizer quality to downstream monolingual performance. For Indian languages, MuRIL [6] and IndicBERT v2 [7] built encoder vocabularies specifically for Indic scripts. IN22 [8] provides professionally translated evaluation data for all 22 scheduled languages, and Dakshina [9] provides human romanizations for South Asian languages. We extend this line of work with current LLM tokenizers, full coverage of the 22 scheduled languages, Indian-built LLM tokenizers, and the romanization and normalization questions.

III. Data and Method

A. Corpora

IN22-Gen [8]: 1,024 general-domain English sentences, each professionally translated into all 22 scheduled languages. Because every language expresses the same content, token counts are directly comparable. We use the openly downloadable MTEB copy of the test split. **Dakshina** [9]: Wikipedia sentences in native script paired with human-written romanizations. We use the 11 Indian languages it covers (about 10,000 sentence pairs each, empty rows removed).

B. Tokenizers

Table I lists the 14 tokenizers: ten used by LLMs and four by encoder models. Krutrim-2's tokenizer has exactly the same vocabulary as Mistral NeMo's (identical counts on every sentence; Krutrim-2 is built on Mistral NeMo [10],

[11]), so the two are reported as one. GPT-4o and GPT-4 are counted with OpenAI's official `tiktoken` [12]. All other tokenizers are loaded from their `tokenizer.json` file with the Hugging Face `tokenizers` library, except MuRIL (WordPiece vocabulary only), which uses `transformers`. Special tokens are never added.

C. Metrics

For tokenizer t and language ℓ, with $n_t(\ell)$ the total number of tokens for the 1,024 sentences, the *token premium* is

$$\pi_t(\ell) = n_t(\ell) \,/\, n_t(\text{English}). \tag{1}$$

A premium of 1 means parity with English. We also report *fertility* (tokens per whitespace-separated word). Encoder vocabularies map unseen characters to a single `[UNK]` token, which makes a text look cheap only because it is discarded. We therefore exclude a (t, ℓ) cell when more than 2% of its tokens are `[UNK]`; this removes 7 languages for mBERT, 3 for XLM-R and 2 for MuRIL. We also record the share of byte-fallback tokens (`<0xNN>`), which signals characters missing from the vocabulary. Confidence intervals are 95% paired bootstrap intervals over sentences (1,000 resamples) [13].

D. Verification

In our first run, GPT-4o's counts were far higher than expected. On a 3,450-sentence check set, the community `tokenizer.json` conversion of GPT-4o's tokenizer produced 466,327 tokens when loaded through `transformers` (v5.22.1), but 319,957 when loaded directly with `tokenizers`, which matched OpenAI's `tiktoken` exactly on all 3,450 sentences. The same discrepancy was 4% for GPT-4. For every other tokenizer, both loading paths gave identical counts. A warning recommending a regex fix flag changed nothing for Mistral NeMo and wrongly altered counts for BERT-style tokenizers, so it was not used. We recommend verifying token counts against an official implementation before reporting them.

IV. Results

A. Overall premium

Among LLM tokenizers, the mean premium ranges from 2.33× (Gemma 3) to 6.69× (GPT-4) (Table I). GPT-4o, the tokenizer behind many deployed applications, averages 3.10×. For an application billed per token, the same content therefore costs roughly three times as much in the average Indian language as in English, and a fixed context window holds roughly a third as much text. The bootstrap intervals are narrow (about ±0.05), so the differences between tokenizers are not sampling noise. The Indic-focused encoder vocabularies reach 1.43× (IndicBERT v2) and 1.34× (MuRIL), which shows that near-parity is achievable: the tax is a design choice rather than a property of the languages.

B. Variation across languages

The language matters more than the tokenizer (Fig. 1). Averaged over tokenizers, the cheapest languages are Hindi (1.98×), Nepali (2.07×) and Marathi (2.14×), all written in Devanagari. The most expensive are Odia (6.1×), Santali in Ol Chiki (9.4×) and Manipuri in Meitei Mayek (9.9×). For the last two, most tokenizers fall back to byte-level tokens, spending several tokens per character. Odia illustrates the effect of vocabulary design: 5.2× with GPT-4o, but 13.1× with Mistral NeMo.

C. Generational change

Replacing GPT-4's tokenizer with GPT-4o's reduced the mean premium from 6.69× to 3.10× (54% lower), and Llama 3 to Llama 4 from 5.68× to 3.45× (39% lower). Fig. 2 shows that the gains are uneven: Telugu fell from 8.0× to 2.0×, but Santali (12.2× →13.2×) and Manipuri (11.3× →11.5×) became slightly more expensive.

D. Indian-built tokenizers

Sarvam-1 targets ten languages (Bengali, Gujarati, Hindi, Kannada, Malayalam, Marathi, Odia, Punjabi, Tamil, Telugu) [16]. On these it is the most efficient LLM tokenizer we tested, at 1.28× versus 1.91× for Gemma 3 and 2.25× for GPT-4o (Fig. 3). On the other twelve scheduled languages its premium rises to 4.08×, above GPT-4o's 3.81×. Perso-Arabic script (Urdu, Kashmiri) falls back almost entirely to bytes. Sarvam reports a fertility of 1.4–2.1 on its target languages [16]. On IN22-Gen we measure 1.5–3.4, with 3 of the ten languages inside the reported range. The difference may reflect text domain or word segmentation, but it shows that efficiency claims depend on the evaluation text. Krutrim-2 inherits Mistral NeMo's premium (3.65× on average).

E. Romanized versus native script

Many users type Indian languages in Latin script. Fig. 4 compares token counts for the same Dakshina sentences in human romanization and native script. For English-centric tokenizers, romanization is much cheaper: 66% fewer tokens with GPT-4, 56% with Llama 3 and 58% with Qwen 3. For GPT-4o the two are nearly equal (ratio 0.99). For Indic-focused tokenizers romanization is more expensive: 1.44× for Sarvam-1, 1.43× for MuRIL and 1.38× for IndicBERT v2. Sarvam-1's low ratios for Urdu and Sindhi arise because these Dakshina languages use Perso-Arabic script, which its vocabulary does not cover. Advice to romanize in order to save tokens is therefore outdated for modern and Indic-focused models.

F. Unicode normalization

Some Indic characters have two canonically equivalent encodings, for example the Bengali letters *yya* and *rra* as single code points or as a base letter plus nukta. Unicode Normalization Form C (NFC) [25] changes 86% of IN22-Gen's Bengali sentences and 70% of its Punjabi sentences. Sarvam-1's vocabulary contains only the decomposed forms, so 95% of its byte fallbacks in Bengali come from these

TABLE I

TOKENIZERS AND MEAN TOKEN PREMIUM OVER THE 22 SCHEDULED LANGUAGES, WITH 95% BOOTSTRAP CI AND THE CHEAPEST AND MOST EXPENSIVE LANGUAGE. *EXCLUDES LANGUAGES WITH >2% [UNK] TOKENS (MBERT 7, XLM-R 3, MURIL 2).

Tokenizer	Developer	Vocab.	Mean premium [95% CI]	Cheapest language	Most expensive language
LLM tokenizers					
Gemma 3 [14]	Google	262k	2.33 [2.30, 2.36]	Bengali (1.18)	Manipuri (Meitei) (8.4)
BLOOM [15]	BigScience	251k	2.48 [2.45, 2.50]	Bengali (1.17)	Santali (Ol Chiki) (12.3)
Sarvam-1 [16]	Sarvam AI	68k	2.81 [2.77, 2.84]	Marathi (1.06)	Santali (Ol Chiki) (10.9)
GPT-4o [17]	OpenAI	200k	3.10 [3.07, 3.13]	Hindi (1.55)	Santali (Ol Chiki) (13.2)
Llama 4 [18]	Meta	201k	3.45 [3.41, 3.49]	Hindi (1.62)	Manipuri (Meitei) (11.4)
Mistral NeMo / Krutrim-2 [10], [11]	Mistral AI / Ola	131k	3.65 [3.61, 3.70]	Urdu (1.70)	Odia (13.1)
DeepSeek-V3 [19]	DeepSeek	129k	3.66 [3.62, 3.70]	Bengali (1.98)	Manipuri (Meitei) (8.3)
Qwen 3 [20]	Alibaba	152k	5.65 [5.57, 5.72]	Urdu (2.95)	Manipuri (Meitei) (11.0)
Llama 3 [21]	Meta	128k	5.68 [5.61, 5.74]	Maithili (2.43)	Odia (12.5)
GPT-4 [22]	OpenAI	100k	6.69 [6.60, 6.77]	Urdu (4.04)	Odia (12.7)
Encoder tokenizers					
MuRIL* [6]	Google	197k	1.34 [1.33, 1.35]	Bengali (1.01)	Bodo (2.0)
IndicBERT v2 [7]	AI4Bharat	250k	1.43 [1.42, 1.45]	Bengali (1.03)	Manipuri (Meitei) (2.9)
XLM-R* [23]	Meta	250k	1.49 [1.47, 1.50]	Nepali (1.14)	Bodo (2.0)
mBERT* [24]	Google	120k	2.18 [2.16, 2.20]	Urdu (1.61)	Assamese (2.7)

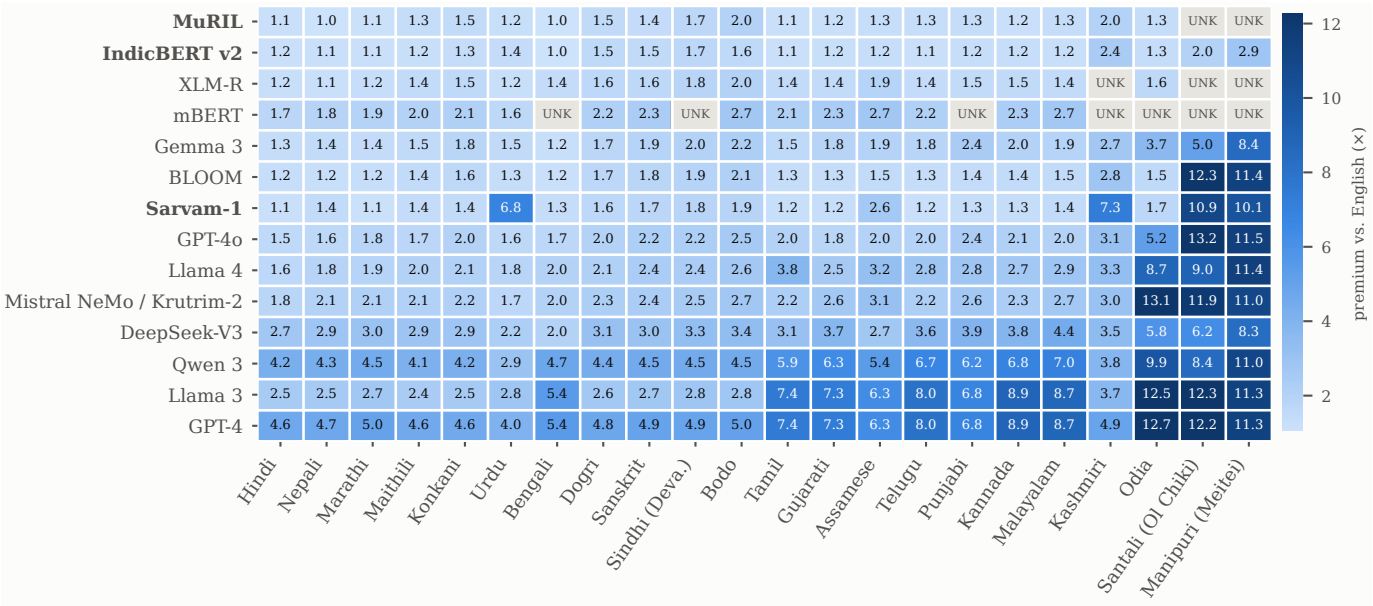


Fig. 1. Token premium relative to English for each tokenizer and language (IN22-Gen). Rows are sorted by mean premium and columns by mean premium across tokenizers. Bold labels mark Indian-built tokenizers. UNK: cell excluded because more than 2% of tokens are [UNK].

two letters. Table II shows that NFC normalization reduces Sarvam-1's Bengali token count by 17.4%, but increases mBERT's by 7.6%. Normalization should match the form a tokenizer was trained on, and both forms should be measured.

TABLE II

TOKEN COUNT CHANGE AFTER NFC NORMALIZATION (LARGEST EFFECTS).

Tokenizer	Language	Change
Sarvam-1	Bengali	−17.4%
Sarvam-1	Punjabi	−10.0%
mBERT	Bengali	+7.6%
Mistral NeMo/Krutrim-2	Bengali	−7.5%
mBERT	Punjabi	+7.3%
mBERT	Sindhi (Deva.)	+5.6%
MuRIL	Punjabi	−4.4%
Sarvam-1	Sindhi (Deva.)	−4.4%

V. DISCUSSION AND LIMITATIONS

For developers serving Indian users, tokenizer choice is a first-order cost decision. Gemma 3 and GPT-4o are the most efficient general-purpose LLM tokenizers here, and Sarvam-1 is the most efficient within its ten languages. Even with the most efficient tokenizers, budgets should assume 2–4× the tokens of an English product, and far more for Odia, Santali or Manipuri. For Indian model builders, the encoder results show that near-parity is achievable, and extending coverage to

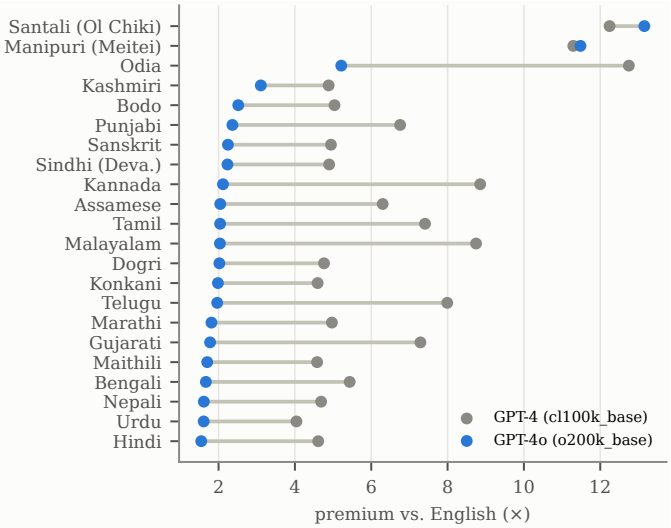


Fig. 2. Premium per language for GPT-4 (cl100k_base) and GPT-4o (o200k_base).

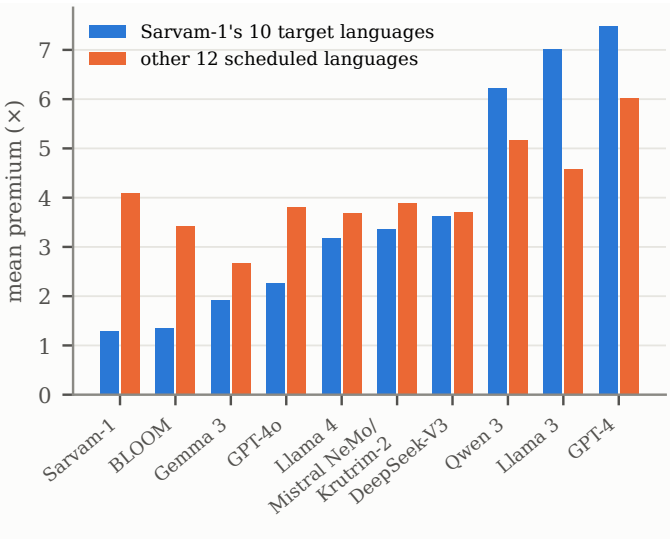


Fig. 3. Mean premium of LLM tokenizers on Sarvam-1's ten target languages versus the other twelve scheduled languages.

all 22 scheduled languages remains the main gap.

Token counts measure cost and context use, not how well a model understands a language. IN22-Gen is formal general-domain text, and conversational, code-mixed or social-media text may tokenize differently. Dakshina romanizations follow Wikipedia-style sentences, while casual romanization varies widely. Premiums are relative to each tokenizer's own English count, so absolute costs also depend on English efficiency and on per-token prices. Several tokenizers were loaded from public mirrors of the official files.

VI. Conclusion

The token tax on Indian languages is large but uneven. Current LLM tokenizers charge the average scheduled language between 2.33× (Gemma 3) and 5.65× (Qwen 3) the tokens

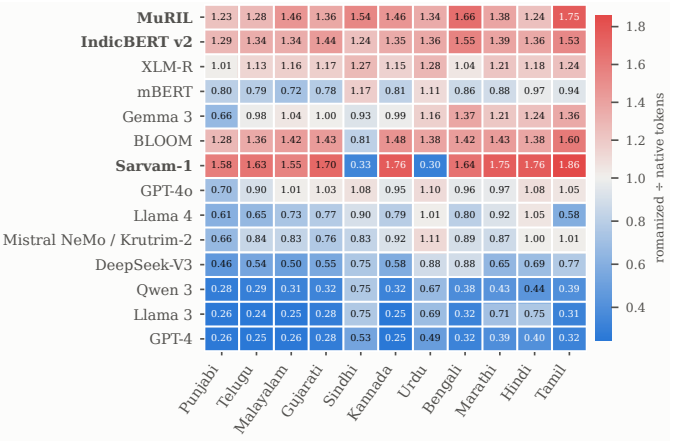


Fig. 4. Ratio of tokens for human-romanized to native-script text (Dakshina). Values below 1 (blue) mean romanized text is cheaper.

of English, newer tokenizers roughly halve older premiums, and the scripts with the least web presence remain an order of magnitude more expensive. Indian-built tokenizers excel within their target languages but do not yet cover all 22. Practical levers exist, including tokenizer choice, script choice and Unicode normalization, but their effect depends on the tokenizer and must be measured.

Data Availability

Code, token counts and figures are available in the Zenodo record of this preprint.

Acknowledgment

The author used Claude (Anthropic), an AI assistant, to help write the analysis code, run the measurements and draft this manuscript. The author is responsible for the content of the paper.

References

[1] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016.

[2] T. Kudo and J. Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," in *Proc. EMNLP: System Demonstrations*, 2018.

[3] A. Petrov, E. L. Malfa, P. H. S. Torr, and A. Bibi, "Language model tokenizers introduce unfairness between languages," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[4] O. Ahia, S. Kumar, H. Gonen, J. Kasai, D. R. Mortensen, N. A. Smith, and Y. Tsvetkov, "Do all languages cost the same? Tokenization in the era of commercial language models," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[5] P. Rust, J. Pfeiffer, I. Vulić, S. Ruder, and I. Gurevych, "How good is your tokenizer? On the monolingual performance of multilingual language models," in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.

[6] S. Khanuja *et al.*, "MuRIL: Multilingual representations for Indian languages," arXiv:2103.10730, 2021.

[7] S. Doddapaneni, R. Aralikatte, G. Ramesh, S. Goyal, M. M. Khapra, A. Kunchukuttan, and P. Kumar, "Towards leaving no Indic language behind: Building monolingual corpora, benchmark and models for Indic languages," in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023, pp. 12 402–12 426.

[8] J. Gala *et al.*, "IndicTrans2: Towards high-quality and accessible machine translation models for all 22 scheduled Indian languages," *Transactions on Machine Learning Research*, 2023.
[9] B. Roark, L. Wolf-Sonkin, C. Kirov, S. J. Mielke, C. Johny, I. Demirşahin, and K. Hall, "Processing South Asian languages written in the Latin script: the Dakshina dataset," in *Proc. Language Resources and Evaluation Conference (LREC)*, 2020.
[10] Ola Krutrim, "Krutrim-2 model card," https://huggingface.co/krutrim-ai-labs/Krutrim-2-instruct, 2025.
[11] Mistral AI, "Mistral NeMo," https://mistral.ai/news/mistral-nemo, 2024.
[12] OpenAI, "tiktoken: a fast BPE tokeniser for use with OpenAI's models," https://github.com/openai/tiktoken, 2023.
[13] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.
[14] Gemma Team, "Gemma 3 technical report," arXiv:2503.19786, 2025.
[15] BigScience Workshop, "BLOOM: A 176B-parameter open-access multilingual language model," arXiv:2211.05100, 2022.
[16] Sarvam AI, "Sarvam-1 model card," https://huggingface.co/sarvamai/sarvam-1, 2024.
[17] OpenAI, "GPT-4o system card," arXiv:2410.21276, 2024.
[18] Meta AI, "The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation," https://ai.meta.com/blog/llama-4-multimodal-intelligence/, 2025.
[19] DeepSeek-AI, "DeepSeek-V3 technical report," arXiv:2412.19437, 2024.
[20] A. Yang *et al.*, "Qwen3 technical report," arXiv:2505.09388, 2025.
[21] A. Grattafiori *et al.*, "The Llama 3 herd of models," arXiv:2407.21783, 2024.
[22] OpenAI, "GPT-4 technical report," arXiv:2303.08774, 2023.
[23] A. Conneau *et al.*, "Unsupervised cross-lingual representation learning at scale," in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.
[24] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019.
[25] K. Whistler, "Unicode normalization forms," The Unicode Consortium, Unicode Standard Annex 15, 2024, https://unicode.org/reports/tr15/.